

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

1.6

MANAGE THE DATABASE

 Copy as Markdown

## Manage the database

Right now your recipes disappear when the Flutter app reloads. Here you'll store them in the database so they persist, and list previously generated recipes in the app. Serverpod maps your model to a table and gives you a type-safe API to read and write rows, without writing any SQL.

Keep `serverpod start` running from the previous page.

### Map the model to a table

Add the `table` keyword to your `Recipe` model in `recipe.spy.yaml`. This maps the model to a database table called `recipes`:

```
### Our AI generated Recipe
class: Recipe
### The database table that stores recipes
table: recipes
fields:
  ### The author of the recipe
  author: String
  ### The recipe text
  text: String
  ### The date the recipe was created
  date: DateTime
  ### The ingredients the user passed in
  ingredients: String
```

Save the file. The regenerated `Recipe` class now exposes database methods through `Recipe.db`.

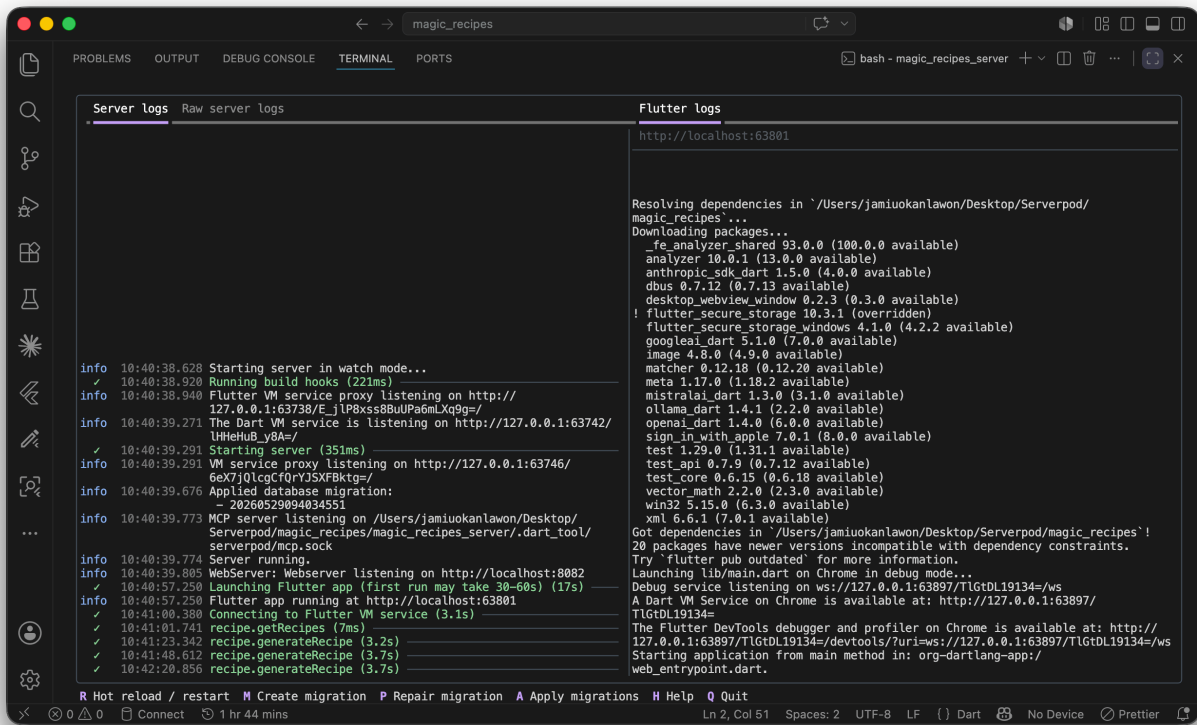
#### INFO

See the [Model reference](#) for all the keywords you can use in a table.



## Create and apply the migration

Changing the schema requires a [migration](#): a set of SQL steps that bring the database up to date with your models. The `serverpod start` terminal has shortcuts for this, listed along the bottom. With that terminal focused:



- Press **M** to create the migration and apply it, which creates the `recipes` table in your database.

## Save recipes to the database

Now that `Recipe` is a table, you can write rows. In `recipe_endpoint.dart`, save the generated recipe before returning it. Replace the `return Recipe(...)` you added earlier with:

```
final recipe = Recipe(  
  author: 'Gemini',  
  text: responseText,  
  date: DateTime.now(),  
  ingredients: ingredients,  
);  
  
return Recipe.db.insertRow(session, recipe);
```

The `insertRow` method returns the saved row with its `id` populated by the database.

## List past recipes

Add a second method to the endpoint that returns every saved recipe, newest first:

```
/// Returns all recipes saved in the database, most recent first.
Future<List<Recipe>> getRecipes(Session session) async {
  return Recipe.db.find(session, orderBy: (t) => t.date.desc());
}
```

### INFO

Both `insertRow` and `find` are Serverpod's typed database methods. See [CRUD](#) for the full set of operations.

## Show the saved recipes in your app

Update `recipe_screen.dart` to load past recipes when it opens and list them next to the generator.

Replace the file with:

```
import 'package:flutter/material.dart';
import 'package:magic_recipe_client/magic_recipe_client.dart';

import '../client.dart';
import 'greetings_screen.dart';

class RecipeScreen extends StatefulWidget {
  const RecipeScreen({super.key});

  @override
  State<RecipeScreen> createState() => _RecipeScreenState();
}

class _RecipeScreenState extends State<RecipeScreen> {
  /// The recipe currently shown, or null if there's none yet.
  Recipe? _recipe;

  /// Recipes loaded from the database, most recent first.
  List<Recipe> _recipeHistory = [];

  /// The last error message, or null if there's no error.
  String? _errorMessage;

  final _textEditingController = TextEditingController();

  bool _loading = false;

  @override
  void initState() {
    super.initState();
    client.recipe.getRecipes().then((recipes) {
```

```

        setState(() => _recipeHistory = recipes);
    });
}

void _callGenerateRecipe() async {
    try {
        setState(() {
            _errorMessage = null;
            _recipe = null;
            _loading = true;
        });

        final result = await client.recipe.generateRecipe(
            _textEditingController.text,
        );

        setState(() {
            _recipe = result;
            _recipeHistory = [result, ..._recipeHistory];
            _loading = false;
        });
    } catch (e) {
        setState(() {
            _errorMessage = '$e';
            _loading = false;
        });
    }
}

@override
Widget build(BuildContext context) {
    return Row(
        children: [
            Expanded(
                child: ListView.builder(
                    itemCount: _recipeHistory.length,
                    itemBuilder: (context, index) {
                        final recipe = _recipeHistory[index];
                        return ListTile(
                            title: Text(recipe.text.split('\n').first),
                            subtitle: Text('${recipe.author} - ${recipe.date}'),
                            onTap: () => setState(() => _recipe = recipe),
                        );
                    },
                ),
            Expanded(
                flex: 3,
                child: Padding(
                    padding: const EdgeInsets.all(16),
                    child: Column(
                        children: [
                            TextField(
                                controller: _textEditingController,
                                decoration: const InputDecoration(
                                    hintText: 'Enter your ingredients',
                                ),
                            ),
                        ],
                    ),
                ),
            ),
        ],
    );
}

```

```

const SizedBox(height: 16),
ElevatedButton(
  onPressed: _loading ? null : _callGenerateRecipe,
  child: _loading
    ? const Text('Loading...')
    : const Text('Generate Recipe'),
),
const SizedBox(height: 16),
Expanded(
  child: SingleChildScrollView(
    child: ResultDisplay(
      resultMessage: _recipe != null
        ? '${_recipe!.author} on ${_recipe!.date}:\n${_recipe!.text}'
        : null,
      errorMessage: _errorMessage,
    ),
  ),
),
),
),
),
),
),
),
),
),
),
),
),
);
}
}

```

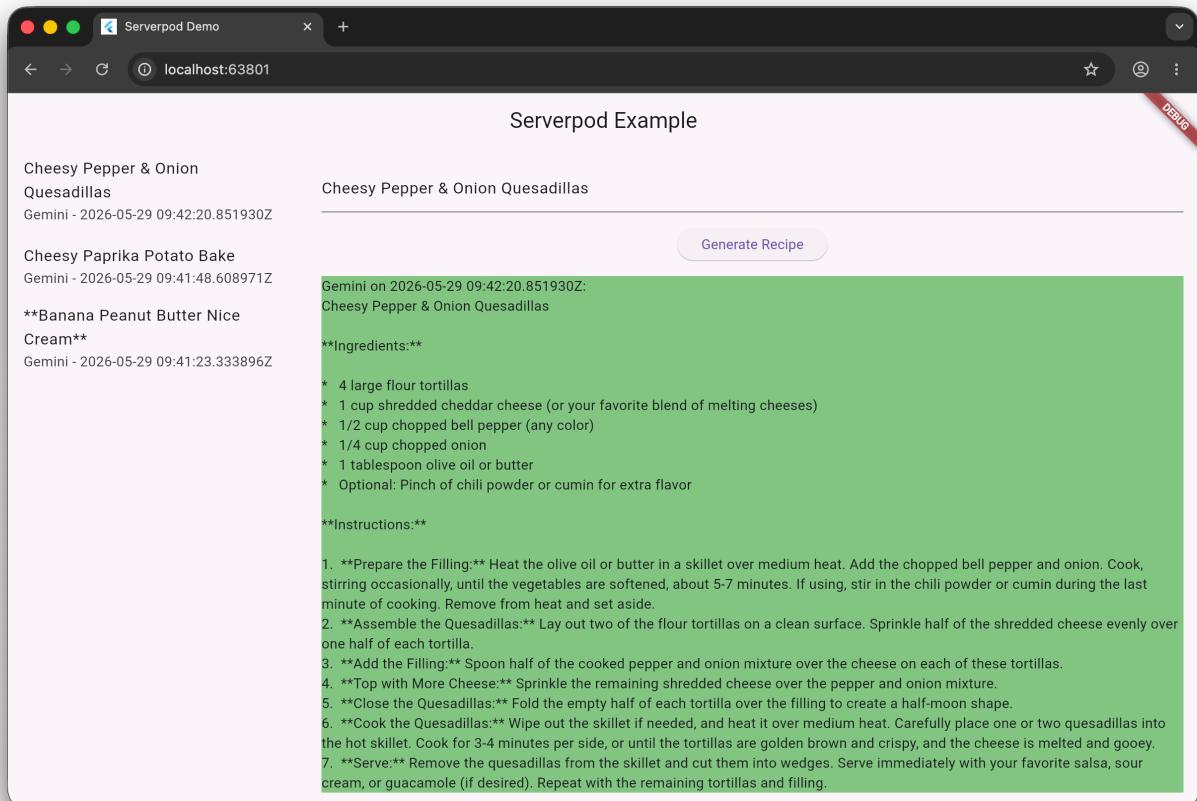
The new `import 'package:magic_recipe_client/magic_recipe_client.dart';` line brings in the `Recipe` class Serverpod generated from your model. It's the same class the server uses, so when you read `recipe.author`, `recipe.text`, or `recipe.date` in the app, the field names and types are guaranteed to match the server.

You added a new endpoint method (`getRecipes`), so the generated client changed.

In the `serverpod start` terminal:

- Press **R** to hot restart.

Generate a few recipes, then reload the page. They're still there, loaded from the database:



## Next steps

Your app now persists data. With the recipe generator working end to end, the last step is to put it online: next, you'll deploy it to Serverpod Cloud.